

ASE2023 Paper #630 Reviews and Comments

=====

Paper #630 Distinguishable Calling Context Encoding

Review #630A

=====

Overall merit

3. Weak accept

Paper summary

This paper presents a clever weight assignment algorithm for runtime calling context encoding and decoding. The authors implemented the algorithm on the LLVM toolchain, and the experimental results show that the proposed algorithm outperforms existing baselines PCCE and DrCCTProf.

Strengths

- + Well-motivated paper
- + Interesting problem
- + Clear presentation
- + Sound approach

Weaknesses

- The system is not fully sound

Comments for authors

This paper presents a runtime calling context encoding and decoding algorithm. The algorithm's core is assigning weights like a numeric system in which lower-level edges receive exponential weights. It appears to be intuitive and sound to me. For me, this is an interesting incremental solution to the problem.

It is a little pity that the system is not fully sound. While dynamic linking seems not a big problem, control flows should be tracked. A potential issue is that call paths may explode, and I speculate that OOM for perlbench, gcc, and imagick reveals the fundamental limitation of this approach. It is a little surprise to me that omnetpp and xalancbmk do not OOM, and this may be due to not accounting for the indirect calls. It would be better to provide more information on the impact of unsoundness.

Finally, this paper is well-written. Dynamic calling context encoding/decoding is not a very popular topic in the software engineering community. I appreciate that the authors well motivated the problem by providing concrete client tools: redundant barrier/lock elision, data flow integrity checking, and context-sensitive code optimization. The comprehensive summary of related work in Section II-B also helped me understand this paper's contributions. I enjoyed reading the formalization: they are concise and clear.

I also have a few thought on this problem. I'm also curious if it is of any interest to use other invertible operations (e.g., multiplication modulo $\$p\$$) to replace "+" in $\$ \$CCID \rightarrow CCID + w, \$ \$$ and $\$w\$$ may not be a single number--to tackle CCID overflow. I'm also curious if it is possible to encode the weight assignment problem and the uniqueness as a constraint and let a constraint solver provide a solution. There might be surprising results, e.g., finding smaller weights.

Review #630B

=====

Overall merit

3. Weak accept

Paper summary

The paper proposes a novel encoding, DCCE, for tracking calling contexts. The advantage of DCCE over existing techniques is that it provides distinguishable and consistent IDs for calling contexts. The central idea is to assign numbers to method invocations based on their depth-first search ordering in the call graph. In addition to providing unique and consistent IDs, the paper shows how DCCE's encoding can be efficiently decoded. The approach is implemented in both static and dynamic tools to compare against static PCCE-style tracking and dynamic DrCCTProf tracking, respectively. Evaluations are performed on two client applications that use the calling context information to aid in their tasks. The data show that DCCE produces comparable overhead encoding with two other approaches and outperforms them in both client applications.

Strengths

- + Straightforward approach
- + Advantage over existing static and dynamic context profiling techniques
- + Evaluation of the proposed technique in two client applications

Weaknesses

- No clear explanation of how recursions are handled
- Some features are not implemented/evaluated in the actual tool
- No checks on the correctness of implementation or how the tool deals with mismatches.
- Experiments are very convoluted, and no actual runtime data is provided

Comments for authors

Significance/Novelty/Soundness/Verifiability/Transparency The paper investigates an important problem of encoding/decoding calling context, which, as the paper describes, has many applications. The paper clearly highlights the advantages of DCCE over two existing approaches: PCCE-style and dynamic-style calling context profilers.

The paper presents a simple yet elegant approach for a novel encoding and clearly explains how it works in an acyclic call graph. However, even though the paper briefly describes how it handles recursions, I still cannot understand how it works – it appears that the tools still cannot determine the depth of the recursion.

The authors performed a thorough evaluation of the proposed technique. In order to make comparisons with existing calling context frameworks, the authors implemented DCCE in static and dynamic tools. In addition, they evaluated both implementations in two different client analyses. However, I found the section on evaluation quite confusing and poorly written. There is no clear rationale for why this set of experiments was selected. Moreover, I find the information presented in the graphs to be too dense to be readable. Another drawback is that evaluation runtime data is only presented as ratios and not the actual data. I believe disclosing the actual runtime is important to fully assess the impact of the proposed technique.

The paper claims that dynamic call profilers, including DrCCTProf cannot guarantee consistent IDs, but the paper does not describe whether it affected the client applications. Also, the paper does not mention how the incomplete implementations of both static and dynamic versions of DCCE are accounted for in the results of the client applications. For example, the paper mentions indirect calls or dynamically linked code.

I feel that the paper focuses more on the instrumentation and the run-time overhead and leaves out discussion on correctness. Do the results produced by two pairs of tools match? I believe these aspects should be discussed in order to provide a full assessment of these approaches. Also, what does DCCE do when it fails to match a context? How often does it happen?

The dynamic version of DCCE seems to be quite complicated and requires significant alteration to both the front and back ends of LLVM. Is it something specific to the proposed techniques, or do other dynamic tools, such as DrCCTProf face similar challenges? Otherwise, even though DrCCTProf is less efficient than Dynamic DCCE, but requires no LLVM modification, then overall, DrCCTProf would be a better choice since it needs less effort for adaptation and maintenance.

Presentation

Overall, I find the paper to be well written. It does contain a few grammatical errors and should be proofread. Also, the presentation of the evaluation section should be better organized; at least provide the reader with rationale on why particular profiling techniques and applications are chosen for comparisons.

Minor:

- * Abstract: DCCE provide -> DCCE provides
- * Introduction, the end of the 1st column: find-grained access -> fine-grained access

Review #630C

Overall merit

2. Weak reject

Paper summary

The paper introduces a new calling context encoding approach named DCCE (Distinguishable Calling Context Encoding). The approach can provide globally distinguishable IDs for different calling contexts across all executions and be used in both static and dynamic analysis. The evaluation shows that DCCE enables calling context using clients to run faster than PCCE and DrCCTProf.

Strengths

- + A new calling context encoding scheme
- + An extensive evaluation of SPEC CPU 2017 and Splash-3

Weaknesses

- Lack of insights behind the approach.
- The paper is not well-written and full of errors, and abbreviations. The paper is hard to read.

Comments for authors

Based on Table I and the evaluation results in Section VI, the paper makes a nice contribution. The proposed approach, DCCE, can distinguish different calling contexts across all executions, making it suitable for both static and dynamic clients. The evaluation shows the approach can enable clients to run faster than existing techniques.

I tend to reject this paper mainly due to its poorly written. The paper is full of abbreviations. Some of them (e.g., IDs, HW, and SW) are used without explaining the meaning. Some of them (PC, CCW, and CCID) are explained by giving the full form but don't appear until much later in the paper. Some (e.g., CC) are introduced but never used later. In the abstraction and Table I, the authors claim that their approach does not need decoding. However, Algorithm 2 presents a decoding algorithm. I have no idea why Algorithm 2 must be presented in the paper. Some pictures presented in the paper (e.g., Fig 1d, and Fig 2d) have never been referred to, which I think they are useless and should be deleted. There are many other writing issues as will be listed below. These writing issues make me hard to judge the correctness of the algorithm. In addition, the paper does not discuss any intuition behind their approach, making it very hard to understand their algorithm.

Below are other issues that I have found when reading this paper.

- * `IDs` appears in the abstract and Section I but are not explained before use.
- * `PC` and `CCIDs` are used in Section I while their meaning or full forms are given much later in the paper.
- * The content in Figure 1 and Figure 2 is not carefully introduced. The authors may assume that readers are already very familiar with this content, which is not true. Introducing the context of the subfigures is not in order. For example, Figure 1c is referenced before Figure 1b.
- * Figure 1d and Figure 2d are not used in the paper. The paper never explains "ID += w ..." in Fig 2b and Fig 2d.
- * In Section 2B, $\rightarrow D$ should not be called a call site.
- * It is unclear to me what the data in `DrCCTProf` column of Fig 2e means.
- * In Section II-C, `p`, `n`, and `ccw` should be in math mode.
- * In Figure 3, $l_{(n, 1)}$ should be $l_{(m, 1)}$?
- * In Equation 1, due to the existence of cycles in Call graphs, the equation does not discuss the case in which the call path from n could be infinitely long.
- * Algorithm 1 is far away from the corresponding describing texts.
- * $\{n_1, \dots, n_3\}$ should be $\{n_1, n_2, n_3\}$.
- * In Figure 4, I could not see why $\$p$ has two call sites that call $\$n_1$ as described in the texts.
- * In Figure 4d-4f, I have no idea how those CCIDs are updated.
- * On page 4, the two paragraphs started with "Definition 6" should be merged.
- * Lemma 1 reads strange. Definition 7, please tell readers what it defines for.
- * I am not sure whether Definition 9 can be called a definition. What is the difference between the back edge in this paper and the one introduced in the compiler book "Compilers: Principles, Techniques, and Tools"?
- * In Algorithm 2, what is $\$E'$? I hope it is not defined in Algorithm 1. If it is indeed defined in Algorithm 1, what is the order for elements in E' there? What is $\$ccw$ in Algorithm 2? Where is it defined? What does $\$callee[n]$ mean?
- * In Section V-A, "on a ASUS M51...", "a" should be "an".
- * "CCEN-PCCE" should be "CCENC-PCCE"?
- * "Table II shows the percentage of edges with non-zero weight". I have no idea what is the weight here for.
- * The table head of Table II is not explained.
- * In Table II, the benchmarks are classified into two groups. In Section VI-B, "Table II displays the benchmarks used in Splash-3". But which group is for Splash-3?

I only list a small part of the written issues of this paper. I hope they can help you improve your paper.

Review #630D

Metareview

The reviewers find the work interesting and novel. However, the presentation of the approach is very weak, which makes it very difficult to read and understand the underlying concept behind the proposed approach. Also, the reviewers are not convinced about the usability of the approach, in particular in the presence of many indirect function calls, which can potentially lead to out-of-memory exceptions.

Recommendation

C. Reject